

Mini-curso Groq Free-tier (4 aulas)

API Free c

01-limites

modelo: qwen/qwen3.6-27b

<think>

1. **Analyze User Input:**

2. **Identify Key Information Needed (from Groq's rate limits doc):**

Actually, Groq's free tier typically has rate limits measured in RPM or TPM, but the user says "Sem inventar RPM". So I'll avoid stating specific RPM numbers. I'll focus on the documented facts: plan names, how rate limits are structured, what happens when limits are exceeded, how to check limits, and the difference between Free and Developer plans according to the doc.

Let me recall/verify typical Groq rate limit docs:

I need to ensure the output is exactly 25-35 lines of markdown, in PT-BR, strictly factual based on the doc, no RPM invented, no API key.

3. **Draft Construction (Mental Refinement - PT-BR):**

Baseado exclusivamente na documentação oficial de rate-limits da Groq

- A Groq oferece dois níveis iniciais: **Free** e **Developer**.

02-429

02-429

modelo: qwen/qwen3.8-27b

****Aula 2 PT-BR: 429/TPM - Retry-After, backoff, o que NÃO fazer****

Quando você recebe um `429 Too Many Requests`, a API está dizendo: "calma, espera um pouco".

Não é bug,

****O que NÃO fazer:****

1. ****Não fa**

****Como fazer direito:****

- Se `Retry

****docs/rate-limits:****

Conforme o

****docs/errors:****

Verifique [o

****Dica final:****

Monitore a
mite.

Total: 32 linhas. ??

03-failover

03-failover

modelo: openai/gpt-oss-20b

****Aula 3 - Failover Multi?Modelo no Free (Qwen/GPT?OSS)****

(25?35 linha

1. O failover multi?modelo garante que, caso um modelo falhe, outro assuma a tarefa.

2. No conte

04-whisper

modelo: openai/gpt-oss-120b

Aula 4 - Whisper ASR One Shot (whisper-large-v3-turbo) vs Chat GPT

1. Nesta aula vamos comparar duas formas de interação com a API da OpenAI: **Whisper ASR one shot** e **Chat GPT**.

04-whisp

*PT?BR - 3

2. **Whisper**
ca requisic

```
python
```

import op

6. O parâmetro `response_format` permite escolher `json`, `text` ou `srt` - útil para legendas?
1?

7. O limite

```
python
```

resp = o

12. O chat aceita **contexto multi-turn**, permitindo que o modelo "lembre" das mensagens anteriores.

13. Quand